Height Balanced Binary Tree

AlgoExpert · Medium · Binary Trees

The problem

- For every node, do left/right subtree heights differ by at most 1?
- Returns -1 from the helper as an "unbalanced" sentinel.

Intuition

- Compute heights bottom-up and fail fast if any node's subtrees differ by more than one.

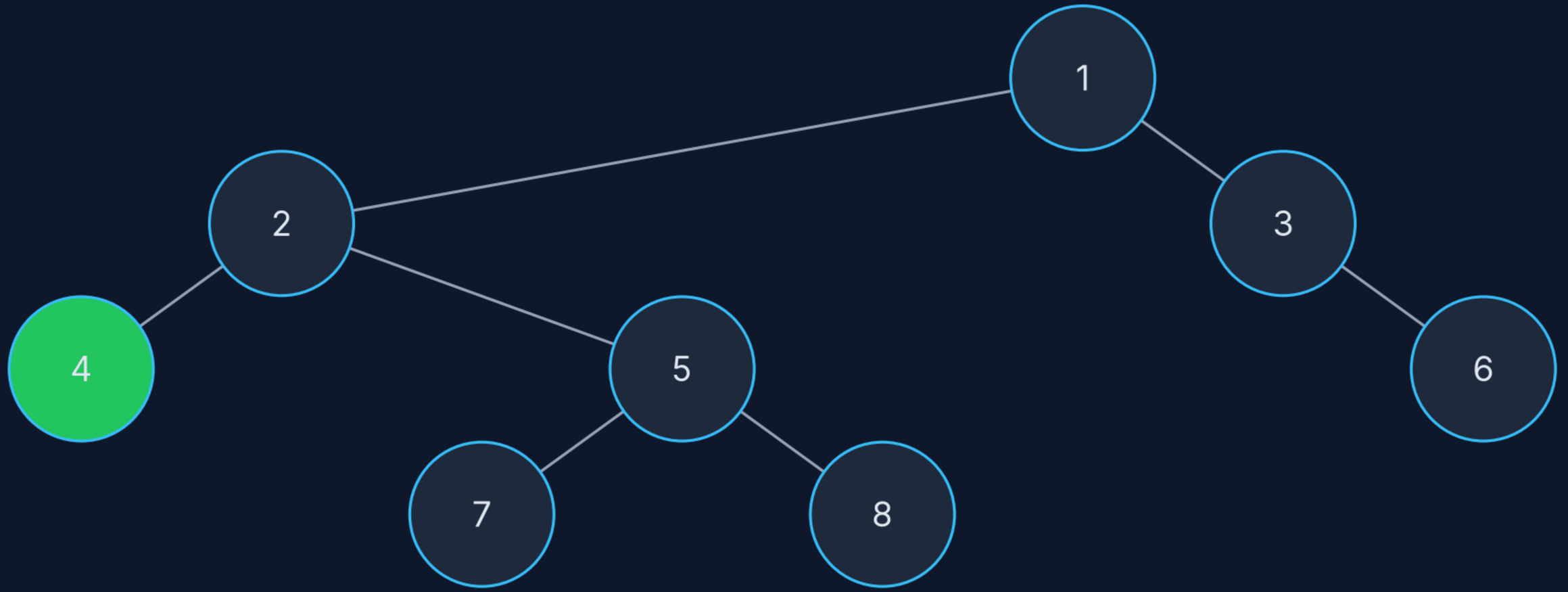
Brute force vs optimal

- Naive: compute height separately at every node → $O(n^2)$
- Single post-order returning height or −1 → $O(n)$
- Fail fast propagates up immediately

Approach

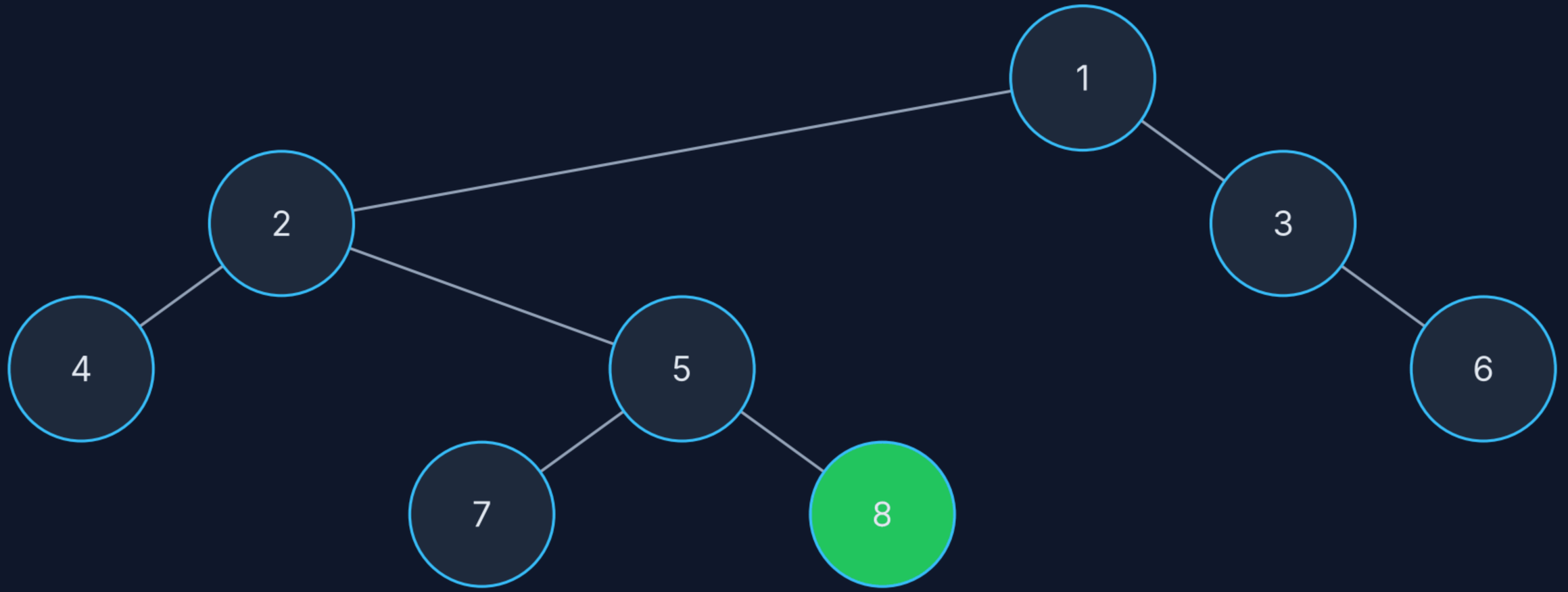
- 1. Post-order returns height and balance
- 2. Compare child heights
- 3. Propagate any imbalance up

Worked example



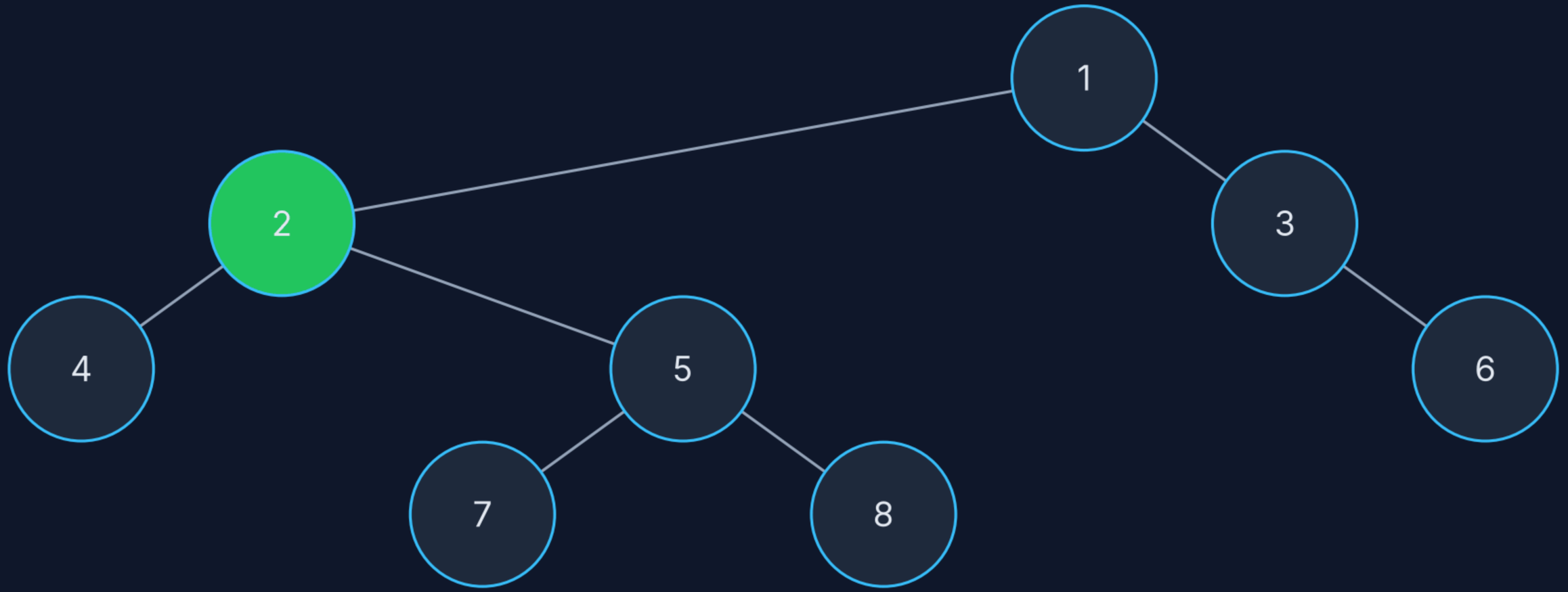
Node 4: heights 0 and 0 → height 1

Worked example



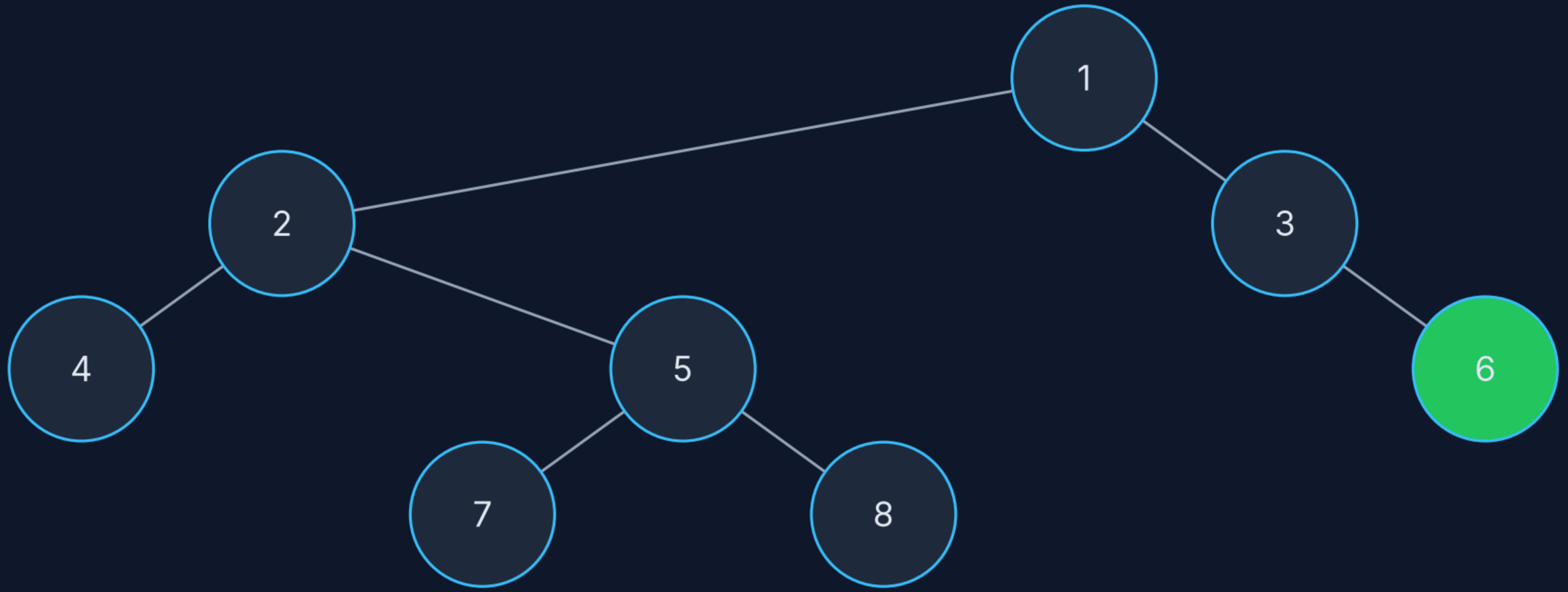
Node 8: heights 0 and 0 → height 1

Worked example



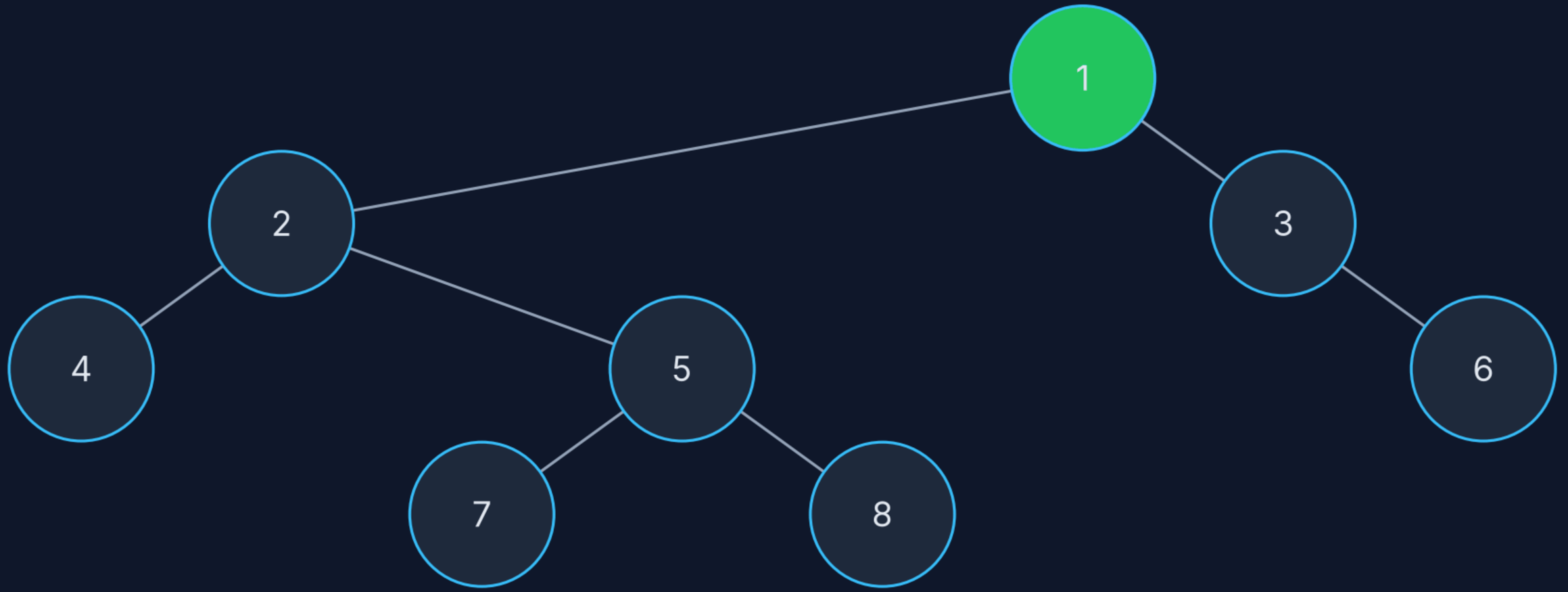
Node 2: heights 1 and 2 → height 3

Worked example



Node 6: heights 0 and 0 → height 1

Worked example



Node 1: heights 3 and 2 → height 4

Java solution

```java
public class HeightBalancedBinaryTree {
    public static boolean heightBalancedBinaryTree(BinaryTree t) { return h(t) != -1; }

    private static int h(BinaryTree t) {
        if (t == null) return 0;
        int l = h(t.left), r = h(t.right);
        if (l == -1 || r == -1 || Math.abs(l - r) > 1) return -1;
        return 1 + Math.max(l, r);
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Edge cases & pitfalls

- Empty tree is balanced
- Check every node, not just the root
- Use a sentinel (−1) to short-circuit

Interview tips & follow-ups

- Follow-up: rebalance via rotations (AVL)
- Return a small object instead of a sentinel

Recap

- Compute heights bottom-up and fail fast if any node's subtrees differ by more than one.
- Complexity: O(n)